



Experiment – 6

Student Name: Riya Mehta

Branch: BE-CSE

Semester: 5th

Subject Name: DAA

UID: 23BCS14042

Section/Group: KRG-2B

Date of Performance: 10/10/25

Subject Code: 23CSH-301

1. **Aim:** Develop a program and analyze complexity to implement subset-sum problem using Dynamic Programming.

2. **Objective:** Objective is to implement subset-sum problem using Dynamic programming.

3. **Input/Apparatus Used:** Standard sum value is taken in order to find subsets of that sum using dynamic programming.

4. Procedure/Algorithm:

Procedure:

So we will create a 2D array of size $(arr.size() + 1) * (target + 1)$ of type boolean. The state $DP[i][j]$ will be true if there exists a subset of elements from $A[0...i]$ with sum value = „j“. The approach for the problem is:

if $(A[i-1] > j)$ $DP[i][j]$

= $DP[i-1][j]$

else

$DP[i][j] = DP[i-1][j] \text{ OR } DP[i-1][j-A[i-1]]$

1. This means that if current element has value greater than „current sum value“ we will copy the answer for previous cases

2. And if the current sum value is greater than the „ith“ element we will see if any of previous states have already experienced the sum=„j“ OR any previous states experienced a value „j



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

– $A[i]$ “ which will solve our purpose.

The below simulation will clarify the above approach: $set[] = \{3, 4, 5, 2\}$

target=6

0 1 2 3 4 5 6

Dynamic Programming – Subset Sum Problem

Given a set of positive integers, and a value *sum S*, find out if there exist a subset in array whose sum is equal to given *sum S*.

Example:

$int[] A = \{3, 2, 7, 1\}, S = 6$

Output: True, subset is (3, 2, 1)

Recursive Approach:

For every element in the array has two options, either we will include that element in subset or we don't include it.

§ So if we take example as $int[] A = \{3, 2, 7, 1\}, S = 6$

§ If we consider another int array with the same size as A.

§ If we include the element in subset we will put 1 in that particular index else put 0.

§ So we need to make every possible subsets and check if any of the subset makes the sum as S.

Time Complexity: $O(2^n)$.

Approach: Dynamic Programming

Base Cases:

§ If no elements in the set then we can't make any subset except for 0.

§ If sum needed is 0 then by returning the empty subset we can make the subset with sum 0.

Given – Set = **arrA[]**, Size = **n**, sum = **S**

§ Now for every element in the set we have 2 options, either we include it or exclude it.

§ for any *i*th element-

§ If include it $\Rightarrow S = S - \text{arrA}[i], n = n - 1$

§ If exclude it $\Rightarrow S, n = n - 1$.

Recursive Equation:

Base Cases:

SubsetSum (arrA, n, S) = false, if sum > 0 and n == 0
SubsetSum (arrA, n, S) = true, if sum == 0 (return empty set)

Rest Cases

SubsetSum (arrA, n, S) = SubsetSum (arrA, n-1, S) || SubsetSum (arrA, n-1, S-arrA[n-1])

How to track the elements.

§ Start from the bottom-right corner and backtrack and check from the True is coming.

§ If value in the cell above is false that means current cell has become true after including the current element. So include the current element and check for the sum = sum – current elements.

Time Complexity: $O(\text{sum} * n)$, where sum is the „target sum“ and „n“ is the size of array.



5. Code and Output:

```
import java.util.*;

public class SubsetSum {

    static int[][] dp = new int[1001][1001];

    static boolean subsetSum(int index, int[] arr, int target) {

        if (target == 0) return true;
        if (index == 0) return (arr[0] == target);

        if (dp[index][target] != -1) return dp[index][target] == 1;

        boolean notPick = subsetSum(index - 1, arr, target);

        boolean pick = false;
        if (arr[index] <= target)
            pick = subsetSum(index - 1, arr, target - arr[index]);

        dp[index][target] = (pick || notPick) ? 1 : 0;
        return pick || notPick;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();

        int[] arr = new int[n];
        System.out.print("Enter elements: ");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        System.out.print("Enter target sum: ");
        int target = sc.nextInt();

        for (int i = 0; i < 1001; i++) {
            Arrays.fill(dp[i], -1);
        }

        if (subsetSum(n - 1, arr, target))
            System.out.println("YES, subset with sum " + target + " exists.");
        else
```



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

```
System.out.println("NO, subset with sum " + target + " does not exist.");
```

```
    sc.close();  
}  
  
}
```

```
Enter number of elements: 5  
Enter elements: 1 0 10 20 50  
Enter target sum: 10  
YES, subset with sum 10 exists.  
PS C:\Users\ASUS\Desktop\Sem 5\DAA_23BCS10889_KRG-2B>
```